



JavaScript — Phase 1 Extended

Deep Dives into Core Concepts

Covering: BigInt, null, undefined, Operators, Functions, and Object.keys/values/entries

Prepared by Claude for Akshay

Table of Contents

1. BigInt — When Numbers Aren't Big Enough
2. null — Detailed Examples
3. undefined — Detailed Examples
4. Operators — Complete Deep Dive
5. Functions — Why, When, and Which Type
6. Object.keys / values / entries

1. BigInt — When Numbers Aren't Big Enough

JavaScript's Number type has a limit. It can only safely handle numbers up to **9,007,199,254,740,991** (about 9 quadrillion). Beyond this, math breaks.

```
console.log(Number.MAX_SAFE_INTEGER);  
// 9007199254740991  
// Beyond this, math breaks:  
console.log(9007199254740991 + 1); // 9007199254740992  
console.log(9007199254740991 + 2); // 9007199254740992 WRONG!  
// Should be 9007199254740993, but JS can't handle it
```

Creating BigInt

Just add n at the end of a number:

```
const big = 9007199254740991n;  
console.log(big + 1n); // 9007199254740992n  
console.log(big + 2n); // 9007199254740993n CORRECT!  
// Two ways to create BigInt  
const way1 = 123456789012345678901234567890n; // literal  
const way2 = BigInt("123456789012345678901234567890"); // from string
```

When Do You Need BigInt?

```
// 1. Database IDs (MongoDB, Twitter snowflake IDs)  
const tweetId = 1234567890123456789n;  
// 2. Financial calculations with large numbers  
const nationalDebt = 31000000000000n; // 31 trillion  
// 3. Cryptography  
const prime = 170141183460469231731687303715884105727n;
```

BigInt Gotchas

```
// Can't mix BigInt and regular numbers!  
10n + 5; // TypeError!  
// Must convert first  
10n + BigInt(5); // 15n  
Number(10n) + 5; // 15  
// No decimals allowed
```

```
10n / 3n; // 3n (truncates, no 3.33...)
// typeof
typeof 10n; // "bigint"
// JSON doesn't support BigInt
JSON.stringify({ id: 10n }); // TypeError!
JSON.stringify({ id: 10n.toString() }); // '{"id":"10"}'
```

Situation	Use Number	Use BigInt
Normal math, prices, scores	Yes	No — overkill
IDs larger than 2^{53}	No — breaks	Yes
Cryptography / security	No	Yes
API with huge IDs	No	Yes

✓ Bottom Line

You probably won't need BigInt daily, but when you hit a bug with large IDs from an API or database, you'll know exactly what to reach for.

2. null — Detailed Examples

null means: "I deliberately set this to nothing." You, the developer, put it there on purpose.

Scenario 1: User Logs Out

```
let currentUser = { name: "Akshay", role: "developer" };  
// User clicks "Logout"  
currentUser = null; // intentionally cleared  
if (currentUser === null) {  
  console.log("Please log in"); // this runs  
}
```

Scenario 2: API Response — Not Found

```
async function getUser(id) {  
  const user = await db.collection("users").findOne({ _id: id });  
  if (user) return user; // found  
  return null;           // not found (intentional)  
}  
const result = await getUser("nonexistent_id");  
if (result === null) {  
  console.log("User not found");  
}
```

Scenario 3: React State — Nothing Selected

```
const [selectedItem, setSelectedItem] = useState(null);  
// null = "nothing selected yet" (deliberate)  
// User picks something:  
setSelectedItem({ id: 1, name: "Laptop" });  
// User deselects:  
setSelectedItem(null); // back to "nothing"
```

Scenario 4: DOM Element Not Found

```
const element = document.getElementById("does-not-exist");  
console.log(element); // null (browser returns null)  
// This is why you see this pattern:  
const btn = document.getElementById("submit-btn");
```

```
if (btn !== null) {  
  btn.addEventListener("click", handleSubmit);  
}  
// Or shorter with optional chaining:  
btn?.addEventListener("click", handleSubmit);
```

Scenario 5: Form Fields — Optional Values

```
const formData = {  
  name: "Akshay",  
  email: "akshay@example.com",  
  middleName: null, // user has no middle name  
  phone: null,      // user chose not to provide  
};
```

Scenario 6: Clearing a Timer

```
let timer = setInterval(() => console.log("tick"), 1000);  
// Later - stop and clean up  
clearInterval(timer);  
timer = null; // signal: timer no longer active
```

Checking for null

```
value === null // strict check (recommended)  
value == null  // catches BOTH null AND undefined  
// Common patterns  
const name = user.name ?? "Anonymous"; // null -> default  
const name = user.name || "Anonymous"; // null, "", 0 -> default
```

3. undefined — Detailed Examples

undefined means: "This hasn't been given a value yet." JavaScript sets this automatically — you almost never set it yourself.

All the Ways undefined Appears

Way 1: Variable declared but not assigned

```
let username;  
console.log(username); // undefined
```

Way 2: Object property that doesn't exist

```
const user = { name: "Akshay", age: 25 };  
console.log(user.email); // undefined  
// This is why you see optional chaining in React:  
console.log(user.address?.city); // undefined (no crash!)  
// Without ?. -> TypeError: Cannot read 'city' of undefined
```

Way 3: Function with no return

```
function doSomething() {  
  console.log("I did something");  
  // no return statement  
}  
const result = doSomething();  
console.log(result); // undefined
```

Way 4: Missing function parameters

```
function greet(name, greeting) {  
  console.log(name); // "Akshay"  
  console.log(greeting); // undefined (not passed!)  
}  
greet("Akshay"); // only passed 1 of 2 parameters
```

Way 5: Array holes / out of bounds

```
const arr = [1, 2, , 4]; // notice the gap  
console.log(arr[2]); // undefined  
const fruits = ["apple", "mango"];
```

```
console.log(fruits[5]); // undefined (no index 5)
```

Way 6: Destructuring missing properties

```
const { name, email } = { name: "Akshay" };  
console.log(name); // "Akshay"  
console.log(email); // undefined
```

Real-World Patterns

Pattern 1: Default parameters

```
function greet(name = "World") {  
  return `Hello, ${name}!`;  
}  
greet(); // "Hello, World!" (undefined triggers default)  
greet("Akshay"); // "Hello, Akshay!"  
greet(null); // "Hello, null!" (null does NOT trigger!)
```

Pattern 2: Checking if a property exists

```
const config = { theme: "dark" };  
// Way A: check for undefined  
if (config.language === undefined) config.language = "en";  
// Way B: 'in' operator (better)  
if (!("language" in config)) config.language = "en";  
// Way C: hasOwnProperty  
if (!config.hasOwnProperty("language")) config.language = "en";
```

Pattern 3: Optional chaining (daily use)

```
const user = { name: "Akshay", address: null };  
user.address?.city // undefined (no crash)  
user.settings?.theme // undefined (no crash)  
user.getProfile?.() // undefined (safe method call)
```

null vs undefined Decision Flowchart

■ How to Decide

Is the value missing?

Did JavaScript set it automatically? -> It's undefined

Variable not assigned, property doesn't exist, parameter not passed, no return

Did YOU intentionally clear it? -> Use null

Logging out, API "not found", clearing a reference, React state "nothing selected"

```
// Never do this - pointless
let x = undefined; // it's already undefined!
// Do this instead
let x;             // same result, cleaner
```

4. Operators — Complete Deep Dive

Arithmetic Operators

```
10 + 3    // 13    addition
10 - 3    // 7     subtraction
10 * 3    // 30    multiplication
10 / 3    // 3.33  division
10 % 3    // 1     remainder (modulo)
10 ** 3   // 1000  exponentiation
```

Modulo (%) — more useful than you think:

```
// Check even/odd
7 % 2     // 1 -> odd
8 % 2     // 0 -> even
// Wrap around (cycling through items)
const colors = ["red", "green", "blue"];
for (let i = 0; i < 10; i++) {
  console.log(colors[i % 3]);
  // red, green, blue, red, green, blue, red...
}
// Clock math
15 % 12   // 3   (3 PM in 12-hour format)
```

Assignment Operators

```
let x = 10;
x += 5;    // x = x + 5 -> 15
x -= 3;    // x = x - 3 -> 12
x *= 2;    // x = x * 2 -> 24
x /= 4;    // x = x / 4 -> 6
x %= 4;    // x = x % 4 -> 2
x **= 3;   // x = x ** 3 -> 8
```

Pre vs Post Increment

```
let a = 5;
// Post-increment: returns THEN increments
let b = a++;
console.log(b); // 5 (got the OLD value)
```

```
console.log(a); // 6 (incremented after)
// Pre-increment: increments THEN returns
let c = 5;
let d = ++c;
console.log(d); // 6 (got the NEW value)
console.log(c); // 6
```

■ Pre vs Post

Post-increment (a++) returns the OLD value then increments. Pre-increment (++a) increments first then returns the NEW value. This is a common interview question!

Comparison: == vs === (The Full Picture)

```
// Loose equality (==) - type coercion happens!
5 == "5"           // true  (string -> number)
0 == false         // true  (false -> 0)
"" == false        // true  (both -> 0)
null == undefined  // true  (special rule)
"0" == false       // true  (both -> 0)
[] == false        // true  ([] -> "" -> 0)
// Strict equality (===) - no coercion
5 === "5"         // false (number vs string)
0 === false       // false (number vs boolean)
null === undefined // false (different types)
```

■ Always Use ===

Loose equality (==) has so many surprising results that it's essentially a bug factory. Even [] == false is true! Always use === except when checking value == null (the one useful exception).

Checking for NaN

```
// NaN is NOT equal to itself!
NaN === NaN           // false
// Use these instead:
Number.isNaN(NaN)     // true
Number.isNaN(5)       // false
Number.isNaN("hello") // false (correct!)
// Old way (avoid - has bugs)
isNaN("hello")        // true (misleading!)
```

Logical Operators — Beyond true/false

```
// && returns FIRST falsy value, or last value
true && "hello"       // "hello"
false && "hello"      // false
"hi" && 0 && "bye"    // 0 (first falsy)
// || returns FIRST truthy value, or last value
false || "hello"     // "hello"
0 || "" || "hi"      // "hi" (first truthy)
// ?? only checks for null/undefined
0 ?? "default"       // 0 (0 is not null/undefined)
"" ?? "default"      // "" (not null/undefined)
null ?? "default"    // "default"
```

```
undefined ?? "default" // "default"
```

|| vs ?? — Critical Difference

```
const userScore = 0;  
// || treats 0 as falsy -> gives WRONG default  
const score1 = userScore || 100;  
console.log(score1); // 100 (user actually scored 0!)  
// ?? only replaces null/undefined -> CORRECT!  
const score2 = userScore ?? 100;  
console.log(score2); // 0 (respects the real score)
```

✓ Rule

Use ?? when 0, "", or false are valid values. Use || when you want to replace ALL falsy values.

String + Number Trap

```
"5" + 3      // "53" (concatenation, NOT addition!)
"5" - 3      // 2   (subtraction converts to number!)
"5" * 3      // 15  (multiplication converts too!)
"5" / 1      // 5   (division converts too!)
// Only + has dual behavior (concat vs addition)
// All other operators (- * / %) always convert to numbers
// Tricky order-of-operations:
5 + 3 + "px"  // "8px" (5+3 first, then string)
"$" + 5 + 3   // "$53" (string from the start)
```

■ The + Trap

This is one of the most common bugs in JavaScript. If ANY operand of + is a string, it becomes concatenation instead of addition. All other math operators (- * / %) always convert to numbers.

Optional Chaining + Nullish Coalescing Together

```
const user = {
  name: "Akshay",
  address: null,
  settings: undefined
};
// The powerful combo:
const city = user.address?.city ?? "Unknown";    // "Unknown"
const theme = user.settings?.theme ?? "light";   // "light"
const email = user.contact?.email ?? "N/A";      // "N/A"
// Without these operators, you'd need:
const city2 = user.address && user.address.city
  ? user.address.city
  : "Unknown";
// So much cleaner with ?. and ??
```

Complete Operator Summary

Category	Operators	Key Point
Arithmetic	+ - * / % **	% for even/odd, cycling
Assignment	= += -= *= /= %=	Shortcuts for math+assign

Comparison	<code>== === != !== > <</code>	Always use <code>===</code>
Logical	<code>&& ! ??</code>	<code>??</code> for null/undefined only
String	<code>+ +=</code>	<code>+</code> with string = concat
Ternary	<code>? :</code>	Short if/else for assignments
Optional Chain	<code>?.</code>	Safe property access

5. Functions — Why, When, and Which Type

Function Declaration — Top-Level Named Functions

```
function calculateTotal(items) {  
  return items.reduce((sum, item) => sum + item.price, 0);  
}
```

When to use: Main functions, functions you want hoisted (callable before definition), recursive functions.

```
// Hoisting - this works!  
greet("Akshay"); // works before definition  
function greet(name) {  
  return `Hello, ${name}!`;  
}  
  
// Recursion - function calls itself by name  
function countdown(n) {  
  if (n <= 0) return;  
  console.log(n);  
  countdown(n - 1);  
}  
  
countdown(5); // 5, 4, 3, 2, 1
```

Arrow Function — Use for Almost Everything Else

```
const double = (n) => n * 2;  
const isEven = (n) => n % 2 === 0;  
const getFullName = (user) => `${user.first} ${user.last}`;
```

When to use: Callbacks (map, filter, reduce), event handlers, React components, async functions, short utilities.

```
// Array methods  
const names = users.map(u => u.name);  
const adults = users.filter(u => u.age >= 18);  
  
// Event handlers  
button.addEventListener("click", () => {  
  console.log("clicked!");  
});  
  
// Async functions  
const fetchUser = async (id) => {
```

```
const res = await fetch(`/api/users/${id}`);
return res.json();
};
// React components
const UserCard = ({ name, email }) => (
  <div>
    <h2>{name}</h2>
    <p>{email}</p>
  </div>
);
```

Function Expression — Rare But Has Its Place

```
// Named expression - good for debugging (stack traces)
const calculate = function calculateTotal(items) {
  return items.reduce((sum, i) => sum + i.price, 0);
};
// As a method on an object (needs its own 'this')
const user = {
  name: "Akshay",
  greet: function() {
    return `Hi, I'm ${this.name}`; // 'this' works
  }
};
```

The Big 'this' Difference

This is the #1 reason to choose between arrow and regular functions:

```
const user = {
  name: "Akshay",
  friends: ["Sara", "John"],
  // Regular function - has its OWN 'this'
  showFriends: function() {
    this.friends.forEach(function(friend) {
      console.log(`${this.name} knows ${friend}`);
      // 'this' is undefined here! BUG!
    });
  },
  // Fixed with arrow - inherits 'this' from parent
  showFriendsFixed: function() {
    this.friends.forEach((friend) => {
      console.log(`${this.name} knows ${friend}`);
      // 'this' still refers to user!
    });
  }
};
```

■ Arrow Functions and 'this'

Arrow functions do NOT have their own this. They inherit it from the surrounding scope. This is why arrows are perfect for callbacks inside methods, but should NOT be used as object methods themselves.

Decision Flowchart

■ Which Function to Use?

Method on an object that uses 'this'? -> Regular function

Callback (map, filter, event, promise)? -> Arrow function

Need hoisting (call before definition)? -> Function declaration

React component? -> Arrow function

General utility? -> Either works, arrow is more common in modern JS

Feature	Declaration	Arrow	Expression
Syntax	function name()	() =>	const x = function()
Hoisted?	Yes	No	No

Own this?	Yes	No (inherits)	Yes
Use as method?	Yes	Avoid	Yes
Callbacks?	Verbose	Perfect	Okay
React components?	Works	Standard	Works
How common in 2026?	Sometimes	Most used	Rare

6. Object.keys / values / entries

These three methods convert an object into an array so you can loop over it, transform it, or use array methods like map, filter, and reduce.

Object.keys() — Get All Property Names

```
const user = { name: "Akshay", age: 25, city: "Pune" };
Object.keys(user);
// ["name", "age", "city"]
```

Real-world uses:

```
// Count properties
Object.keys(user).length; // 3
// Check if object is empty
Object.keys({}).length === 0; // true
// Check if a key exists
Object.keys(user).includes("email"); // false
// Loop over keys
Object.keys(user).forEach(key => {
  console.log(`${key}: ${user[key]}`);
});
```

Object.values() — Get All Property Values

```
const scores = { math: 85, science: 92, english: 78 };
Object.values(scores); // [85, 92, 78]
// Sum all values
const total = Object.values(scores).reduce((sum, s) => sum + s, 0);
// 255
// Find maximum
const highest = Math.max(...Object.values(scores));
// 92
// Check if any value meets condition
Object.values(scores).some(s => s > 90); // true
```

Object.entries() — Get [key, value] Pairs

This is the **most powerful** of the three because you get both key AND value:

```
const user = { name: "Akshay", age: 25, city: "Pune" };
Object.entries(user);
// [["name", "Akshay"], ["age", 25], ["city", "Pune"]]
// Loop with both key and value
Object.entries(user).forEach(([key, value]) => {
  console.log(`${key}: ${value}`);
});
```

Transform an Object (entries + fromEntries)

```
// Apply 10% discount to all prices
const prices = { laptop: 1200, phone: 699, tablet: 499 };
const discounted = Object.fromEntries(
  Object.entries(prices).map(([item, price]) => [item, price * 0.9])
);
// { laptop: 1080, phone: 629.1, tablet: 449.1 }
```

Filter an Object

```
// Remove null and undefined values
const data = { name: "Akshay", age: 25, email: null, phone: undefined };
const cleaned = Object.fromEntries(
  Object.entries(data).filter(([key, value]) => value != null)
);
// { name: "Akshay", age: 25 }
```

Convert Object to Query String

```
const params = { search: "laptop", page: 1, sort: "price" };
const queryString = Object.entries(params)
  .map(([key, value]) => `${key}=${value}`)
  .join("&");
// "search=laptop&page=1&sort=price"
```

The Power Pattern: Object Transform Pipeline

```
const inventory = {
  laptop: 15, phone: 0, tablet: 8, charger: 0, mouse: 23
};
```

```
// "Get items in stock, sorted by quantity"
const inStock = Object.entries(inventory)
  .filter(([item, qty]) => qty > 0)
  .sort((a, b) => b[1] - a[1])
  .map(([item, qty]) => `${item}: ${qty}`);
// ["mouse: 23", "laptop: 15", "tablet: 8"]
```

Quick Summary

Method	Returns	Use When
Object.keys(obj)	Array of keys	Count props, check empty, loop keys
Object.values(obj)	Array of values	Sum, find max, check conditions
Object.entries(obj)	[key, value] pairs	Transform, filter, loop with both
Object.fromEntries(arr)	Object from array	Build back an object

✓ When to Use Which

Need **only keys**? Use `Object.keys()` (check if empty, count props). Need **only values**? Use `Object.values()` (sum, find max). Need **both key and value**? Use `Object.entries()` (transform, filter). Need to **build back** an object? Use `Object.fromEntries()`.